

5) Extend Vehicle

Add encapsulated fields:

- id : String (unique)
- mileage : int
- rented : boolean

Add methods:

- public void rent()
 - throws error if already rented
- public void returnVehicle(int drivenKm)
 - throws error if not rented
 - drivenKm must be > 0
 - increases mileage, sets rented=false

Add abstract methods (abstraction + polymorphism):

- public abstract boolean needsService();
- public abstract double rentalPrice(int days);

6) Override per subclass

Service rules:

- Car: service every **10,000 km**
- Motorcycle: every **6,000 km**
- Truck: every **15,000 km**

Pricing rules:

- Car: $50 * \text{days}$, +10% if $\text{doors} \geq 4$
- Motorcycle: $30 * \text{days}$, +15/day if hasSidecar
- Truck: $(80 + 0.02 * \text{loadCapacity}) * \text{days}$

7) Create Garage (array-based manager)

Fields:

- private Vehicle[] fleet;
- private int size;

Methods:

- add(Vehicle v) → fails if full OR duplicate id
- findById(String id) → returns Vehicle or null
- rentById(String id)
- returnById(String id, int drivenKm)
- printAvailable()
- printNeedsService()
- printRentalEstimate(String id, int days)

8) Create Demo (array-based manager):

Create a garage of capacity 10; Add at least 5 vehicles; Rent 2 of them;